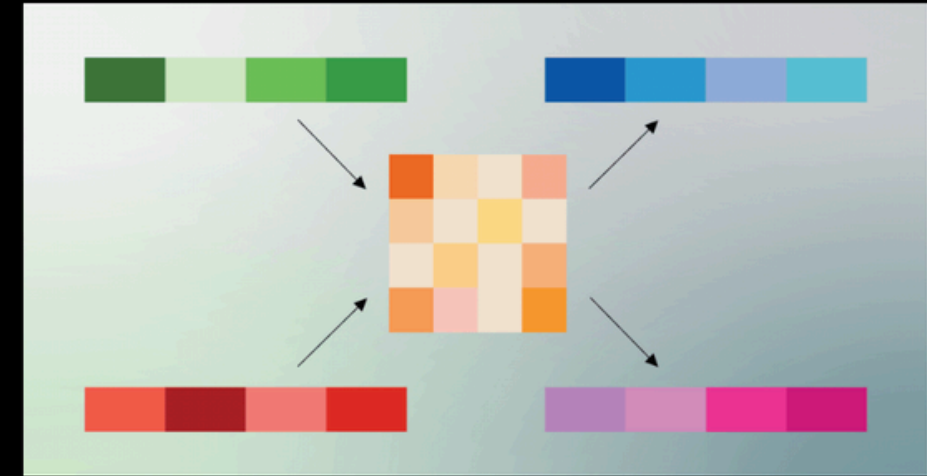


Week 7

[Home](#) > [Blog](#) >

TurboQuant: Redefining AI efficiency with extreme compression

March 24, 2026 · Amir Zandieh, Research Scientist, and Vahab Mirrokni, VP and Google Fellow, Google Research



Vector => PolarQuant + QJL => Compressed Vector

Traditional compression holds vector in cartesian coordinates

=>

Memory overhead

PolarQuant uses polar coordinate: radius and angle
the distribution of the angles are known, no overhead

PolarQuant gives an estimate of Key, Value Vectors

This estimate is biased

The bias accumulates across many **attention** layers

QJL is an algorithm that corrects the bias with 1-bit

The estimator is now mathematically unbiased

We first validated the core algorithm on random unit vectors against the paper's theoretical bounds.

MSE Distortion (d=128, 1000 random unit vectors):

Bits	Measured MSE	Paper's Upper Bound	Ratio
1-bit	0.362	0.680	0.53x
2-bit	0.116	0.170	0.68x
3-bit	0.034	0.043	0.81x
4-bit	0.009	0.011	0.87x

<https://github.com/tonbistudio/turboquant-pytorch>

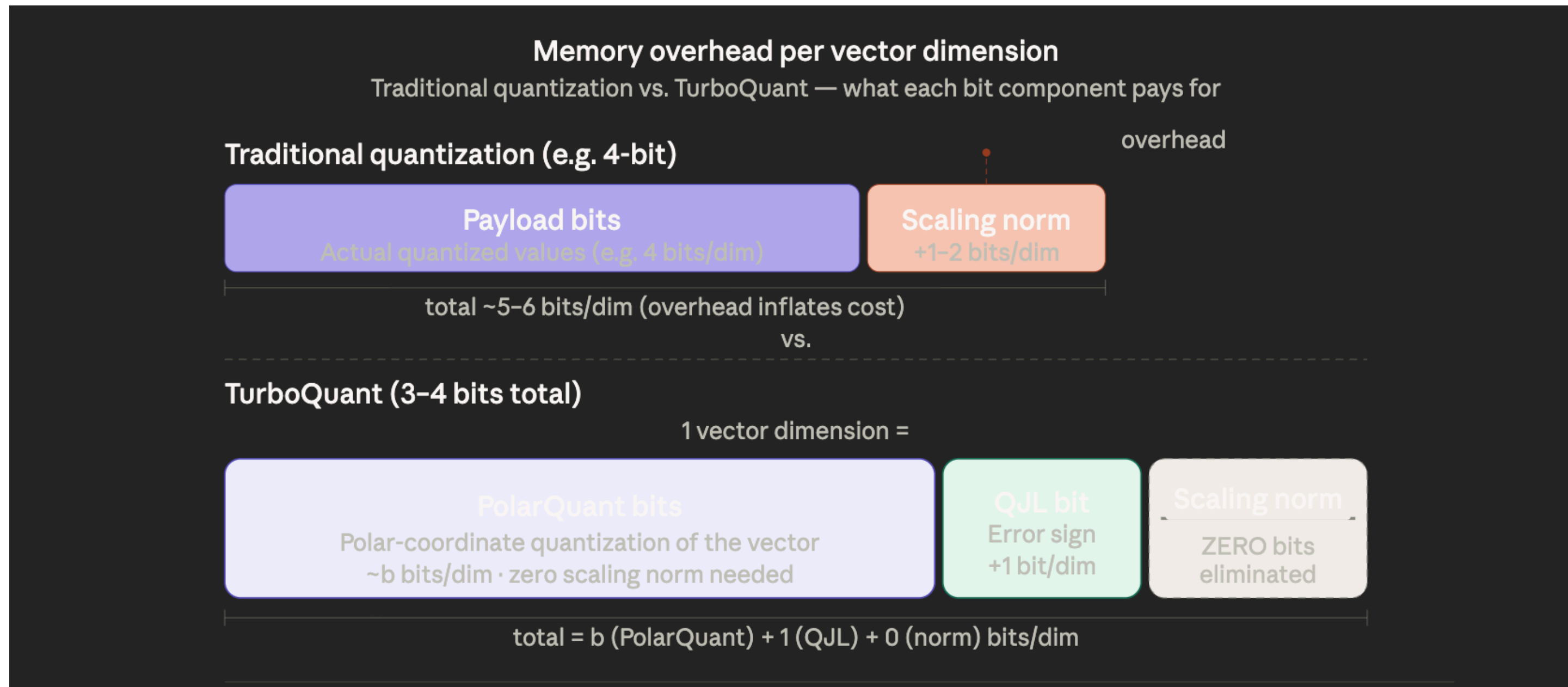
TurboQuant actually distorts the vectors, but the inner products of them Q.K remains very close

Compression Ratios (consistent across all context lengths):

Config	KV Cache Size (8K ctx)	Compression
FP16 (baseline)	289 MB	1.0x
TurboQuant 4-bit	76 MB	3.8x
TurboQuant 3-bit	58 MB	5.0x
TurboQuant 2-bit	40 MB	7.3x

Attention Score Accuracy (averaged across all 36 layers, 2 KV heads per layer = 72 total checks):

Config	Context	Cosine Sim	Top-1 Match	Top-5 Match
TQ-4bit	2K	0.9989	85%	96%
TQ-4bit	4K	0.9986	92%	94%
TQ-4bit	8K	0.9983	86%	96%
TQ-3bit	2K	0.9961	85%	94%
TQ-3bit	4K	0.9955	75%	88%
TQ-3bit	8K	0.9945	86%	94%
TQ-2bit	2K	0.9897	63%	83%
TQ-2bit	4K	0.9878	65%	85%
TQ-2bit	8K	0.9851	71%	89%



traditional quantization needs training (k-means) for
designing codebook
turboquant calculates everything on the fly

My test with a retrieval task from MTEB

Dataset: SciFact, nearly 5k docs, 1k queries

Embedding: Snowflake/snowflake-arctic-embed-s:
33M params, small

